

Chapter 3: Microprocessor Memory Organization — Questions and Answers

Q3.1: What is the basic difference between main memory and secondary memory?

Main memory (Primary memory) is the memory that the CPU can access directly. It is fast but smaller in size. It stores the programs and data that are currently running. Examples are RAM and ROM.

Secondary memory is used to store large amounts of data at a low cost. The CPU cannot access it directly. It is slower than main memory. Examples are hard disks, CDs, and DVDs.

Feature	Main Memory	Secondary Memory
Access by CPU	Direct	Indirect (via OS)
Speed	Fast	Slow
Size	Smaller	Larger
Cost	More expensive	Less expensive
Examples	RAM, ROM	Hard disk, CD, DVD

Q3.2: A microprocessor has 24 address pins. What is the maximum size of the main memory?

When a microprocessor has 24 address pins, it can generate 2^{24} unique addresses.

$$2^{24} = 16,777,216 = 16 \text{ MB}$$

So the maximum size of the main memory is 16 MB (16 Megabytes).

Q3.3: Can the microprocessor execute programs directly in hard disk? Explain your answer.

No. The microprocessor cannot execute programs directly from the hard disk.

The CPU can only access main memory (RAM and ROM) directly using its address bus. Hard disk is secondary memory, and the CPU has no direct connection to it.

Before a program can run, the operating system must first load it from the hard disk into main memory (RAM). Only then can the CPU fetch and execute the instructions.

Also, hard disk is much slower than main memory, so executing directly from it would be extremely slow even if it were possible.

Q3.4: What is the basic difference between: (a) EPROM and EEPROM? (b) SRAM and DRAM?

(a) EPROM vs EEPROM:

Feature	EPROM	EEPROM
Erasure method	Ultraviolet (UV) light for 10–20 minutes	Electrically, without removing chip
Chip removal needed?	Yes, must be removed to erase/program	No, can be erased in-circuit
Read speed vs Write speed	Fast read	Read is faster than write (read-mostly)
Example use	Older embedded systems	Modern embedded systems

(b) SRAM vs DRAM:

Feature	SRAM	DRAM
Storage element	Flip-flops	Capacitors
Refresh needed?	No	Yes, every few milliseconds
Speed	Faster	Slower
Cost	More expensive	Cheaper
Cell size	Larger	Smaller
Use	Small, fast memory (cache)	Large main memory

Q3.5: Given a memory with a 14-bit address and an 8-bit word size.

(a) How many bytes can be stored in this memory?

A 14-bit address can address $2^{14} = 16,384$ memory locations.

Each location stores 8 bits = 1 byte.

Total = 16,384 bytes = 16 KB

(b) If this memory were constructed from 1K x 1 RAMs, how many memory chips would be required?

Total bits needed = 16,384 locations x 8 bits = 131,072 bits

Each 1K x 1 chip stores = $1024 \times 1 = 1024$ bits

Number of chips = $131,072 / 1024 = 128$ chips

(c) How many bits would be used for chip select?

We have 128 chips in groups of 8 (for 8-bit word width), so there are $128 / 8 = 16$ groups.

To select 16 groups: $2^4 = 16$, so 4 bits are used for chip select.

(These are the upper 4 address bits: A10–A13 out of 14 bits, since lower 10 bits select within a 1K chip.)

Q3.6: What are the main differences between CD and DVD memories?

Feature	CD	DVD
Laser type	Infrared laser	Red laser (shorter wavelength)
Storage capacity	Less (about 700 MB)	More (4.7 GB or higher)
Track pitch	Wider	Narrower (allows more data)
Usage	Audio, basic data	Video, large data, software

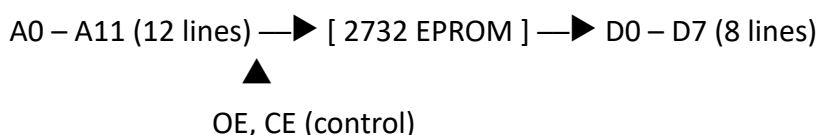
The shorter wavelength of the DVD laser allows it to read smaller pits and tracks, which is why it can store much more data than a CD.

Q3.7: Draw a block diagram showing the address and data lines for the 2732, and 2764 EPROM chips.

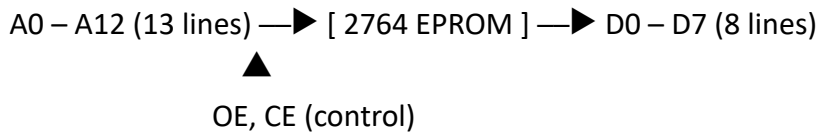
The 2732 is a 4K x 8 EPROM (4096 locations, 8-bit data). It needs 12 address lines (A0–A11) and 8 data lines (D0–D7).

The 2764 is an 8K x 8 EPROM (8192 locations, 8-bit data). It needs 13 address lines (A0–A12) and 8 data lines (D0–D7).

Block diagram (2732 EPROM):



Block diagram (2764 EPROM):



Q3.8 (a): How many address and data lines are required for a 1M x 16 memory chip?

1M = 2^{20} , so 20 address lines are needed (A0–A19).

16-bit data width means 16 data lines (D0–D15).

Answer: 20 address lines and 16 data lines.

Q3.8 (b): What is the size of a decoder with one chip enable (CE bar) to obtain a 64K x 32 memory from 4K x 8 chips? Where are the inputs and outputs connected?

Step 1 – How many chips are needed?

Total bits = $64K \times 32 = 65,536 \times 32 = 2,097,152$ bits

Each chip = $4K \times 8 = 4096 \times 8 = 32,768$ bits

Number of chips = $2,097,152 / 32,768 = 64$ chips

Step 2 – Grouping chips:

Each group provides 32-bit width. Each chip is 8-bit, so $32/8 = 4$ chips per group.

Number of groups = $64 / 4 = 16$ groups

Step 3 – Decoder size:

We need to select 1 out of 16 groups. So the decoder is a 4-to-16 decoder (with one CE bar input).

Inputs: High-order address bits (A12–A15) go to the decoder inputs.

Outputs: Each output goes to the CE bar (chip enable) of one group of 4 chips.

Q3.9: A microprocessor with 24 address pins and eight data pins is connected to a 1K x 8 memory chip with one chip enable. How many unused address bits are available for interfacing other 1K x 8 chips? What is the maximum directly addressable memory?

The microprocessor has 24 address lines (A0–A23).

A 1K x 8 chip needs 10 address lines (A0–A9) to select one of 1024 locations.

Unused address bits = $24 - 10 = 14$ bits (A10–A23).

These 14 unused bits can be used as chip selects for other 1K x 8 chips.

Maximum directly addressable memory = $2^{24} = 16,777,216$ bytes = 16 MB.

Q3.10: Name the methods used in main memory array design. What are the advantages and disadvantages of each?

The two main methods are:

1. Linear (Direct) Decoding

- How it works: High-order address bits directly act as chip select signals. No decoder chip is used.
- Advantage: Simple circuit — no extra hardware needed.
- Disadvantage 1: Bus conflicts may occur if two high-order bits are LOW at the same time, selecting two chips together.
- Disadvantage 2: Memory foldback (wasted address space) — some addresses map to the same memory location.

2. Full (Complete) Decoding

- How it works: A decoder chip (like 74LS138) is used. It takes high-order address bits as input and activates exactly one chip select output at a time.

- Advantage 1: No bus conflicts — only one chip is selected at a time.
- Advantage 2: No wasted address space — each address maps to exactly one location.
- Disadvantage: Requires extra hardware (decoder chip).

Q3.11: The block diagram of a 512 x 8 RAM chip is shown in Figure P3.11. The memory chip is enabled only when CS1 bar = L and CS2 = H. Design a 1K x 8 RAM system using this chip as the building block. Draw a neat logic diagram. Assume the microprocessor can directly address 64K with R/W bar and eight data pins. Using linear decoding and don't-care conditions as 1's, determine the memory map in hexadecimal.

Step 1 – How many chips needed?

We need $1K \times 8 = 1024 \times 8$ total. Each chip is 512×8 .

Number of chips = $1K / 512 = 2$ chips.

Step 2 – Connection:

Both chips share the same 9 lower address lines (A0–A8) and 8 data lines (D0–D7).

Address line A9 is used to select between the two chips:

- When A9 = 0: Chip 1 is selected (addresses 0000H–01FFH in the chip's range).
- When A9 = 1: Chip 2 is selected (addresses 0200H–03FFH in the chip's range).

Step 3 – Logic diagram connections:

A0–A8 —————> Both chips (address inputs)

A9 —> NOT gate —> CS1 bar of Chip 1 (A9=0 → CS1 bar=LOW → Chip 1 enabled)

A9 —————> CS2 of Chip 2 (A9=1 → CS2=HIGH → Chip 2 enabled)

CS2 of Chip 1 = HIGH (tied to Vcc)

CS1 bar of Chip 2 tied LOW (always LOW)

WE bar from CPU —> WE bar of both chips

Step 4 – Memory Map (using linear decoding with don't-cares = 1):

The microprocessor has 16 address lines (A0–A15) for 64K addressing.

Lower 10 bits (A0–A9) used for chip addressing. Upper 6 bits (A10–A15) are don't-cares = 1.

Chip	A15–A10 (don't care=1)	A9	A8–A0	Address Range (Hex)
Chip 1	111111	0	000000000	FC00H – FDFFH
Chip 2	111111	1	000000000	FE00H – FFFFH

Q3.12: Consider the hardware schematic shown in Figure P3.12. (a) Determine the address map. (b) Is there any possibility of bus conflict?

From Figure P3.12, the system contains a 2K x 8 ROM and a 256 x 8 RAM connected to a 16-bit address microprocessor via a 3-to-8 decoder.

(a) Address Map:

The decoder inputs are connected to A11, A12, A13 (high-order bits). M/IO bar must be HIGH (memory operation). MEMR bar controls read.

The 2K x 8 ROM uses address lines A0–A10 (11 lines). The 256 x 8 RAM uses address lines A0–A7 (8

lines).

Based on which decoder output is connected:

Device	Size	Approximate Address Range
2K x 8 ROM	2048 bytes	Depends on decoder output connected to OE bar
256 x 8 RAM	256 bytes	Depends on decoder output connected to CS bar

The exact ranges depend on which outputs (Y0–Y7) of the 74138 decoder connect to the ROM and RAM.

(b) Bus Conflict:

No, there is no bus conflict. The 74138 decoder ensures that only one output is LOW at a time. This means only one chip is selected at any given time. So the ROM and RAM cannot drive the data bus simultaneously.

Q3.13: Interface a microprocessor with 16-bit address pins and 8-bit data pins and a R/W bar pin to a 1K x 8 EPROM chip and two 1K x 8 RAM chips to obtain the memory map in Table P3.13. Use a 74138 decoder.

Target memory map (from Table P3.13):

Device	Size	Address (Hex)
EPROM chip	1K x 8	8000–83FF
RAM chip 0	1K x 8	9000–93FF
RAM chip 1	1K x 8	C000–C3FF

Step 1 – Address analysis:

Each chip is 1K = 1024 locations, so 10 address lines (A0–A9) go to each chip directly.

The upper 6 bits (A10–A15) are used for decoding with the 74138.

Step 2 – 74138 decoder (3-to-8):

Connect A13, A14, A15 to inputs A, B, C of the 74138.

G1 = HIGH, G2A bar = LOW (tied), G2B bar = driven by A12 bar (active when A12 = 0).

Step 3 – Chip connections:

- EPROM (8000H–83FF): A15=1, A14=0, A13=0 → decoder output Y4 (active LOW) → connect to CE bar of EPROM. OE bar of EPROM connected to MEMR bar (read signal). A10–A11 = 00.
- RAM chip 0 (9000H–93FF): A15=1, A14=0, A13=0, A12=1 → decoder output Y4 with A12 differentiation → connect output to CE of RAM 0. WE bar connected to R/W bar of CPU.
- RAM chip 1 (C000H–C3FF): A15=1, A14=1, A13=0 → decoder output Y6 → connect to CE of RAM 1.

A0–A9 of CPU connect to A0–A9 of all three chips. Data lines D0–D7 connect to all chips (only one active at a time via decoder).

Q3.14: Repeat Problem 3.13 to obtain the memory map shown in Table P3.14 using a 74138 decoder.

Target memory map (from Table P3.14):

Device	Size	Address (Hex)
EPROM chip	1K x 8	7000–73FF
RAM chip 0	1K x 8	D000–D3FF
RAM chip 1	1K x 8	F000–F3FF

Step 1 – Address analysis:

Again, lower 10 bits A0–A9 go to each chip. Upper bits A10–A15 used for decoding.

Step 2 – Decode the addresses:

7000H = 0111 0000 0000 0000 → A15=0, A14=1, A13=1, A12=1

D000H = 1101 0000 0000 0000 → A15=1, A14=1, A13=0, A12=1

F000H = 1111 0000 0000 0000 → A15=1, A14=1, A13=1, A12=1

Step 3 – 74138 decoder connections:

Connect A13, A14, A15 to inputs A, B, C of 74138. Use A12 and A11 for additional chip enable control.

- EPROM (7000H): A15=0, A14=1, A13=1 → output Y3 of decoder → CE bar of EPROM
- RAM chip 0 (D000H): A15=1, A14=1, A13=0 → output Y6 → CE of RAM 0, WE bar = R/W bar
- RAM chip 1 (F000H): A15=1, A14=1, A13=1 → output Y7 → CE of RAM 1, WE bar = R/W bar

A0–A9 connect to all chips. D0–D7 bus shared across all chips.

Q3.15: What is meant by foldback in linear decoding?

Foldback (also called memory foldback) is a problem that happens in linear decoding.

In linear decoding, the high-order address bits are used directly as chip select signals without using a decoder. The lower bits select a location inside the chip.

The high-order bits that are not used for chip select are simply ignored. This means that multiple different addresses in the CPU's address space all point to the same physical memory location inside the chip.

Example: If a 1K chip uses A0–A9 and A10 is the chip select, then addresses like 0000H and 0400H both access the same chip (if A10 is ignored). This is called foldback or memory mirroring.

Effect: It wastes address space. The same memory appears at multiple address locations.

Q3.16: Comment on the importance of the following features in an operating system implementation: (a) Address translation, (b) Protection.

(a) Address Translation:

Programs use logical (symbolic) addresses, but the actual hardware uses physical addresses in RAM.

The MMU (Memory Management Unit) translates logical addresses to physical addresses. This is very important because:

- It allows programs to be loaded anywhere in physical memory without changing their code.
- It supports virtual memory — programs can use more memory than physically available.
- It allows relocation of programs and sharing of memory between processes.
- Methods: Substitution (direct mapping) or Offset (adding a base address).

(b) Protection:

Protection prevents one program from reading or writing into the memory area of another program or the operating system.

This is important because:

- It keeps the OS safe from user programs.
- It prevents bugs in one program from corrupting another program's data.
- The MMU checks the privilege level of the CPU against the privilege level of the memory segment. If the CPU privilege is lower, access is denied and an interrupt is generated.
- This makes the system stable and secure.

Q3.17: Explain briefly the differences between segmentation and paging.

Feature	Paging	Segmentation
Block size	Equal-sized pages	Variable-sized segments
Fragmentation type	Internal fragmentation (unused space inside a page)	External fragmentation (gaps between segments)
Address structure	Page number + offset	Segment number + offset
Management	Easier — all blocks same size	More complex — variable sizes
Advantage	Eliminates external fragmentation	Matches program's logical structure

Paging divides memory into fixed equal-sized blocks called pages. Segmentation divides memory into variable-sized blocks called segments that match the logical structure of a program (like code, data, and stack).

Modern systems often combine both — called paged segmentation — where each segment is divided into pages.

Q3.18: What is the advantage of having a cache memory? Name a 32-bit microprocessor that does not contain an on-chip cache.

Advantage of cache memory:

Cache memory is a small but very fast memory placed between the CPU and main memory. Its main advantage is that it speeds up program execution.

Programs often use the same memory locations repeatedly (called locality of reference). By keeping these frequently used instructions and data in the cache, the CPU can access them much faster than going to slower main memory.

- Main memory access time: about 500 ns
- Cache access time: about 100 ns

This reduces the number of slow main memory accesses and improves overall performance.

32-bit microprocessor without on-chip cache:

The Intel 80386 does not have an on-chip cache. It uses an external cache memory chip.

Q3.19: What basic functions are performed by a microprocessor's on-chip MMU?

The on-chip MMU (Memory Management Unit) performs two basic functions:

1. Address Translation:

The MMU converts logical addresses (used by programs) to physical addresses (actual RAM locations).

Two methods are used:

- Substitution: Direct replacement of logical address with physical address — fast.
- Offset: The MMU adds an offset (base address) to the logical address to get the physical address — more flexible.

2. Memory Protection:

The MMU protects memory areas from unauthorized access. Each memory segment has a privilege level. The MMU compares the CPU's privilege level with the segment's privilege level:

- If CPU privilege is high enough: access is allowed.
- If CPU privilege is too low: the MMU generates an interrupt and blocks the access.

This prevents user programs from accessing OS memory or each other's memory areas.

Q3.20: Discuss briefly the various cache-mapping techniques.

There are three cache-mapping techniques:

1. Direct Mapping:

- Each main memory block maps to exactly one specific cache line.
- The memory address is split into: Tag + Index + Word.
- Index selects the cache line. Tag checks if the right block is there.
- Simple and fast, but may cause frequent misses if two blocks compete for the same line.

2. Fully Associative Mapping:

- Any memory block can be placed in any cache line.
- The cache stores both the full address (tag) and the data.
- The CPU searches all cache entries at the same time.
- Very flexible and has fewer misses, but is expensive due to complex hardware.

3. Set-Associative Mapping:

- A combination of direct and fully associative mapping.
- Cache is divided into sets. Each set can hold multiple blocks.
- Index selects the set. Tags are compared within the set associatively.
- Better hit ratio than direct mapping, and cheaper than fully associative.
- Replacement policies used: FIFO or LRU.

Q3.21: A microprocessor has a main memory of 8K x 32 and a cache memory of 4K x 32. Using direct mapping, determine the sizes of the tag field, index field, and each word of the cache.

Given:

- Main memory: 8K x 32 → 8192 words, each 32 bits wide
- Cache memory: 4K x 32 → 4096 words, each 32 bits wide

Step 1 – Address size for main memory:

$8K = 2^{13}$, so main memory address = 13 bits.

Step 2 – Index field size:

Cache has $4K = 2^{12}$ lines, so index field = 12 bits.

Step 3 – Tag field size:

Tag = Total address bits – Index bits = 13 – 12 = 1 bit.

Step 4 – Each cache word:

Each cache word contains: 1 tag bit + 32 data bits = 33 bits.

Field	Size
Tag field	1 bit
Index field	12 bits
Each cache word	33 bits (1 tag + 32 data)

Q3.22: A microprocessor has a main memory of 4K x 32. Using a cache memory address of 8 bits and set-associative mapping with a set size of 2, determine the size of the cache memory.

Given:

- Main memory: 4K x 32 → 4096 words, 32 bits each
- Cache memory address: 8 bits → $2^8 = 256$ sets (index)
- Set size: 2 (each set holds 2 memory words)

Step 1 – Total main memory address bits:

4K = 2^{12} , so 12 address bits for main memory.

Step 2 – Tag field size:

Tag = 12 – 8 = 4 bits.

Step 3 – Size of each cache word:

Each set holds 2 words. Each word = 32 data bits + 4 tag bits = 36 bits.

Each set = 2 x 36 = 72 bits.

Step 4 – Total cache size:

256 sets x 72 bits = 18,432 bits = 2304 bytes.

The cache can store 256 x 2 = 512 memory words total.

Parameter	Value
Number of sets (index)	256
Set size	2 words per set
Tag field	4 bits
Each word in cache	36 bits (32 data + 4 tag)
Total cache size	18,432 bits (512 words x 36 bits)

Q3.23: A microprocessor can address directly 1 MB of memory with a 16-bit word size. Determine the size of each cache memory word for associative mapping.

Given:

- Main memory: 1 MB = 1,048,576 bytes = 2^{20} bytes
- Word size: 16 bits = 2 bytes
- Number of 16-bit words in memory: $2^{20} / 2 = 2^{19} = 524,288$ words

Step 1 – Main memory address size:

2^{19} words $\rightarrow$ 19 address bits needed.

Step 2 – Associative mapping:

In fully associative mapping, the cache stores both the full address (tag) and the data word for each entry. There is no index field — the entire address is the tag.

Step 3 – Size of each cache word:

Each cache word = Tag (full address) + Data word = 19 bits + 16 bits = 35 bits.

Component	Size
Tag (full memory address)	19 bits
Data word	16 bits
Each cache memory word	35 bits

Q3.24: Under what conditions does the set-associative mapping method become one of the following? (a) Direct mapping, (b) Fully associative mapping.

Set-associative mapping is a general method. It can become either extreme depending on the set size:

(a) Becomes Direct Mapping:

When the set size = 1 (each set holds only 1 block), set-associative mapping becomes direct mapping. In this case, each memory block maps to exactly one cache line — which is the definition of direct mapping.

(b) Becomes Fully Associative Mapping:

When the set size = total number of cache lines (only 1 set for the whole cache), set-associative mapping becomes fully associative mapping.

In this case, any memory block can go into any cache line — which is the definition of fully associative mapping.

Set Size	Equivalent Mapping
1 (one block per set)	Direct Mapping
= Total cache lines (one set only)	Fully Associative Mapping
Between 1 and total lines	Set-Associative Mapping